



Transaction Processing Pipeline

Student Name: Yevhen Kulinichenko AAI02

Date Submitted: 06.07.2026

AI Tools Used: Claude Code

A file-based, three-stage Node.js transaction pipeline
validation → fraud detection → settlement



Problem / Objective

“

Build a Node.js/Express-compatible, file-based transaction processing pipeline that validates, fraud-scores, and settles the transactions in

`sample-transactions.json` in sequence (validator → fraud detector →

settlement), orchestrated end-to-end and reported on in

`shared/results/`.

”

SOURCE

`specification.md` — High-Level Objective



Architecture



Pipeline Stages

Three sequential stages, each a pure function over a JSON envelope

1 Validator — `pipeline/validator.js`

Checks required fields, parses `amount` with `decimal.js` (never `Number` / `parseFloat`), validates currency against an ISO 4217 allow-list

(`USD`, `EUR`, `GBP`, `JPY`), and enforces the refund sign rule.

✗ **TXN006** — currency
"XYZ", \$200 transfer
Rejected: `invalid_currency`
— not on the allow-list.

✓ **TXN007** — refund,
`amount: "-100.00"`
Normalized via
`Decimal.abs()` → `"100.00"`,
marked `validated`.

2 Fraud Detector —

`pipeline/fraud-detector.js`

Sums five weighted risk factors (0–100, capped) against every non-rejected transaction:

Factor	Condition	Weight
<code>high_value</code>	<code>amount > \$10k</code>	+40
<code>very_high_value</code>	<code>amount > \$50k</code>	+20
<code>unusual_timing</code>	<code>00:00–06:00 UTC</code>	+20
<code>cross_border</code>	<code>metadata.country !== "US"</code>	+15
<code>wire_transfer</code>	<code>transfer method</code>	+10

🚩 Fraud Scoring Example

“
TXN005 — \$75,000 wire transfer, US, 10:00 UTC
`high_value` (+40) + `very_high_value` (+20) + `wire_transfer`
(+10) = **70**
→ `flagged_fraud`
”

Design note: TXN002 scores exactly 50 by the same literal formula — `reviewed`, not flagged — resolving an ambiguity between the spec's prose and its own worked arithmetic in favor of the arithmetic.

3 Settlement — `pipeline/settlement.js`

Turns the fraud detector's `status` into a `final_status`:

-  `rejected` **stays** `rejected` (reason preserved)
-  `flagged_fraud` **becomes** `held_for_review` (never auto-settled)
-  `validated` / `reviewed` **becomes** `settled` (+ `settled_at` timestamp)

“

TXN001 — \$1,500 USD transfer, US, business hours → scores 0

→

`validated` → `settled`, written to `shared/results/TXN001.json`

”



The Four-Agent Workflow

#	Agent	Role
1	<code>spec-writer</code>	Writes <code>specification.md</code> before any code
2	<code>pipeline-codegen</code>	Builds <code>orchestrator.js</code> , <code>pipeline/*.js</code> , <code>frontend/</code> ; uses context7
3	<code>unit-test-writer</code>	Jest suite + coverage gate (80% threshold)
4	<code>docs-writer</code>	README, HOWTORUN, PR description, this deck

Defined in `.claude/agents/*.md`, each with a scoped `tools:` allow-list —

no agent invents requirements outside `specification.md` / `TASKS.md`.

MCP Integration

context7 — used by `pipeline-codegen` before non-trivial library usage.

“
"Decimal constructor invalid string error handling, isNaN, abs(), greaterThan comparison" → resolved to `/mikemcl/decimal.js` → confirmed
`new Decimal('abc')` *throws* rather than returning `NaN`.
”

Custom `pipeline-status` **server** (`mcp/server.js`, built on `@modelcontextprotocol/sdk`):

Skills & Hooks

- `/run-pipeline` — clears `shared/{input,processing,output,results}/`, runs `node orchestrator.js`, renders results as markdown tables
- `/validate-transactions` — read-only dry-run validation, no `shared/` writes
-  **Coverage-gate hook** (`PreToolUse` → `scripts/check-coverage.js`) — blocks `git push` when line coverage is below **80%**



Express front-end

```
npm run frontend → :3000
```

Triggers a fresh run, renders
summary + per-transaction
table in-browser

The screenshot shows a web application titled "Transaction Processing Pipeline". It includes a sub-header "Runs all sample transactions through validations, fraud detection, and settlement." and a link "Run Pipeline" in a blue button. Below this, a summary section displays four metrics: 8 TOTAL, 6 SETTLED, 1 REJECTED, and 1 HELD FOR REVIEW. A table below the summary lists transaction details with columns for Transaction ID, Fraud Status, and Reason. The table contains 8 rows of data, with some entries highlighted in orange (TXN005) or green (TXN006).

Transaction ID	Fraud Status	Reason
TXN001	verified	
TXN002	verified	
TXN003	verified	
TXN004	verified	
TXN005	held for review	
TXN006	verified	invalid currency
TXN007	verified	
TXN008	verified	

CLI pipeline run

```
npm run pipeline
```

Prints a human-readable summary with counts per transaction outcome

[illegible]

Metric	Coverage
Statements	95.07% (193/203)
Branches	87.17% (68/78)
Functions	100% (18/18)
Lines	94.94% (188/198)

29 tests · 4 suites · all passing — well above the 80% coverage-gate threshold.



Lessons Learned

- **decimal.js** beats native **Number** — but *throws* on unparseable input instead of returning **NaN**; context7 caught this before it crashed
- **File-based message passing** makes every hand-off inspectable on disk, at the cost of extra I/O
- **Spec ambiguity must be resolved in code, not silently**
- **Scoped subagent tool allow-lists** kept each agent honest about its job

Thank you 🙌

Questions welcome